ML Numerical by MPS

1. Information Gain

Formula:

$$\text{Information Gain} = \text{Entropy(Parent)} - \sum \left(\frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v) \right)$$

Entropy:

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i)$$

Steps:

1. Calculate Entropy of the parent dataset.
2. Split the dataset by attribute values.
3. Calculate Entropy for each subset.
4. Use the formula above to compute gain.

Example:
Dataset: 9 Yes, 5 No

$$\text{Entropy} = -\frac{9}{14} \log_2 \left(\frac{9}{14}\right) - \frac{5}{14} \log_2 \left(\frac{5}{14}\right)$$

2. Design a Spam Filter using Bayesian Classification

Formula:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Steps:

1. Calculate prior $P(\text{Spam})$, $P(\text{Not Spam})$
2. Calculate likelihood $P(\text{word}|\text{Spam})$
3. Multiply all word probabilities and compare posterior probabilities.

Tip: Use Laplace smoothing:

$$P(w|C) = \frac{\text{Count}(w \in C) + 1}{\text{Total words in C} + |Vocab|}$$

3. Neural Network Backpropagation

Key Equations:

- Error:

$$E = \frac{1}{2}(y_{true} - y_{pred})^2$$

- Weight Update Rule:

$$w := w - \eta \cdot \frac{\partial E}{\partial w}$$

Steps:

1. Forward pass: compute outputs.
2. Compute error at output.
3. Backpropagate error using chain rule.
4. Update weights using learning rate η.

---

4. Confusion Matrix Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1\text{-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

---

5. Boosting (AdaBoost)

Weight Update Rule:

- Error:

$$\text{Error}_t = \sum w_i \cdot I(h_t(x_i) \neq y_i)$$

- Alpha:

$$\alpha_t = \frac{1}{2} \log\left(\frac{1 - \text{Error}_t}{\text{Error}_t}\right)$$

- Weight update:

$$w_i := w_i \cdot e^{-\alpha_t y_i h_t(x_i)}$$

Steps:

1. Assign equal weights initially.
2. Train weak classifier.
3. Update weights → focus on misclassified points.

4. Combine weak learners with weighted votes.

---

6. Equation of Regression Line (Given Data Points)

$$y = mx + c$$

Where:

$$m = \frac{n(\sum xy) - (\sum x)(\sum y)}{n(\sum x^2) - (\sum x)^2}$$

$$c = \frac{\sum y - m \sum x}{n}$$

Steps:

1. Compute $\sum x, \sum y, \sum x^2, \sum xy$
2. Plug into formulas for m and c.

---

7. Distance from Hyperplane – SVM

Given hyperplane: $w \cdot x + b = 0$
Distance of point x_0:

$$\text{Distance} = \frac{|w \cdot x_0 + b|}{||w||}$$

---

8. Logistic Regression

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{where } z = w \cdot x + b$$

Loss (Log Loss):

$$L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

Gradient Descent Updates:

$$w := w - \eta \cdot \frac{\partial L}{\partial w}$$

---

9. Data Preprocessing Techniques

- Missing Values: Mean/Median/Mode, KNN, or Drop.
- Scaling:
  - Min-Max:

$$x' = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

- Z-Score:

$$x' = \frac{x - \mu}{\sigma}$$

- Encoding:
  - Label Encoding
  - One-Hot Encoding
- Outlier Detection: IQR, Z-score

---

10. CNN Concepts – Striding, Padding, etc.

- Convolution Output Size:

$$O = \frac{(W - K + 2P)}{S} + 1$$

Where:

- W: Input size
- K: Kernel size
- P: Padding
- S: Stride
- Padding:
  - Valid: No padding
  - Same: Output size = Input size
- Pooling (Max/Average): Reduces spatial size, helps in translation invariance

---